

Experiment / Case Study No.: 5

1. Experiment Title:

Apply Array Methods and Object Handling and Create a Cart Total Calculator with Discount Logic

2. Software/Tools Required:

Visual Studio Code, HTML5, CSS3, JavaScript (Array Methods — push, filter, forEach, reduce — and Object Handling), Google Chrome Browser

3. Experiment Program Code:

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Shopping Cart Calculator | Soumya Suryawanshi</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="main-container">
    <header>
      <h1>Shopping Cart Calculator</h1>
      <p>Manage your purchases efficiently</p>
    </header>

    <!-- Input Section -->
    <div class="card input-card">
      <h2>Add New Product</h2>
      <div class="input-group">
        <input type="text" id="productName" placeholder="Product Name
(e.g., Pen, Laptop)">
        <input type="number" id="price" placeholder="Price (₹)" min="0"
step="0.50">
        <input type="number" id="quantity" placeholder="Quantity" min="1"
value="1">
        <button onclick="addProduct()" class="btn-add">Add Product</button>
      </div>
    </div>

    <!-- Table Section -->
    <div class="card table-card">
      <h2>Cart Items</h2>
      <div class="table-wrapper">
        <table>
          <thead>
            <tr>
              <th>ID</th>
              <th>Product Name</th>
              <th>Price (₹)</th>
```

```

                <th>Qty</th>
                <th>Total (₹)</th>
                <th>Action</th>
            </tr>
        </thead>
        <tbody id="cartBody">
            <!-- Rows injected by JS -->
        </tbody>
    </table>
</div>
</div>

<!-- Summary Section -->
<div class="card summary-card">
    <div class="totals-grid">
        <div class="total-box">
            <span class="label">Total Amount</span>
            <span class="value" id="totalAmount">₹0.00</span>
        </div>
        <div class="total-box discount">
            <span class="label">Discount (20%)</span>
            <span class="value" id="discountAmount">-₹0.00</span>
        </div>
        <div class="total-box final">
            <span class="label">Final Amount</span>
            <span class="value" id="finalAmount">₹0.00</span>
        </div>
    </div>
</div>

<div class="lists-grid">
    <!-- Low Cost List -->
    <div class="list-box">
        <h3>Low-Cost Items (Price < ₹1000)</h3>
        <ul id="lowCostList"></ul>
        <p id="noLowCost" class="empty-msg">No low-cost items
found.</p>
    </div>

    <!-- Expensive List -->
    <div class="list-box">
        <h3>Expensive Items (Price > ₹1000)</h3>
        <ul id="expensiveList"></ul>
        <p id="noExpensive" class="empty-msg">No expensive items
found.</p>
    </div>
</div>

<div class="item-summary-box">
    <h3>Item Summary</h3>
    <ul id="itemSummary" style="margin-top: 10px;"></ul>
</div>
</div>

<div class="footer">
    <span>Soumya Suryawanshi</span>

```

```
        <span>PRN: 24070521062</span>
    </div>
</div>

<script src="script.js"></script>
</body>
</html>
```

script.js

```
// Global State
let cart = [];
let idCounter = 1;

// Helper function to format numbers as Indian Rupee (INR)
const formatINR = (amount) => {
    return '₹' + amount.toLocaleString('en-IN', {
        minimumFractionDigits: 2,
        maximumFractionDigits: 2
    });
};

// Add Product
function addProduct() {
    const nameInput = document.getElementById('productName');
    const priceInput = document.getElementById('price');
    const qtyInput = document.getElementById('quantity');

    const name = nameInput.value.trim();
    const price = parseFloat(priceInput.value);
    const qty = parseInt(qtyInput.value);

    // Validation
    if (name === "") {
        alert("Please enter a product name.");
        nameInput.focus();
        return;
    }
    if (isNaN(price) || price <= 0) {
        alert("Please enter a valid price greater than 0.");
        priceInput.focus();
        return;
    }
    if (isNaN(qty) || qty < 1) {
        alert("Please enter a valid quantity (at least 1).");
        qtyInput.focus();
        return;
    }

    const total = price * qty;

    // Object handling: each cart entry is an object
    const product = {
```

```

        id: idCounter++,
        name: name,
        price: price,
        qty: qty,
        total: total
    });

    cart.push(product); // Array method: push

    // Clear inputs
    nameInput.value = "";
    priceInput.value = "";
    qtyInput.value = "1";
    nameInput.focus();

    renderCart();
    calculateTotals();
}

// Delete Product
function deleteProduct(id) {
    if (confirm("Are you sure you want to delete this item?")) {
        cart = cart.filter(item => item.id !== id); // Array method: filter
        renderCart();
        calculateTotals();
    }
}

// Render Table
function renderCart() {
    const tbody = document.getElementById('cartBody');
    tbody.innerHTML = "";

    if (cart.length === 0) {
        tbody.innerHTML = `
            <tr>
                <td colspan="6" style="text-align:center; color:#999; padding:
20px;">
                    No products added yet. Try adding a "Pen" (₹10) or "Laptop"
(₹30000).
                </td>
            </tr>
        `;
        return;
    }

    cart.forEach(item => { // Array method: forEach
        const row = document.createElement('tr');
        row.innerHTML = `
            <td>${item.id}</td>
            <td>${item.name}</td>
            <td>${formatINR(item.price)}</td>
            <td>${item.qty}</td>
            <td style="font-weight: 600;">${formatINR(item.total)}</td>
            <td>

```

```

        <button class="btn-delete" onclick="deleteProduct(${
item.id})">Delete</button>
      </td>
    `;
    tbody.appendChild(row);
  });
}

// Calculate Totals and Render Lists
function calculateTotals() {
  // 1. Total Amount – Array method: reduce
  const totalAmount = cart.reduce((sum, item) => sum + item.total, 0);

  // 2. Discount (20%)
  const discountRate = 0.20;
  const discountAmount = totalAmount * discountRate;

  // 3. Final Amount
  const finalAmount = totalAmount - discountAmount;

  // Update UI
  document.getElementById('totalAmount').innerText = formatINR(totalAmount);
  document.getElementById('discountAmount').innerText = "-" +
formatINR(discountAmount);
  document.getElementById('finalAmount').innerText = formatINR(finalAmount);

  renderLists();
}

// Render Lists (Low Cost, Expensive, Summary)
function renderLists() {
  const lowCostList = document.getElementById('lowCostList');
  const expensiveList = document.getElementById('expensiveList');
  const noLowCostMsg = document.getElementById('noLowCost');
  const noExpensiveMsg = document.getElementById('noExpensive');
  const itemSummaryList = document.getElementById('itemSummary');

  lowCostList.innerHTML = "";
  expensiveList.innerHTML = "";
  itemSummaryList.innerHTML = "";

  let hasLowCost = false;
  let hasExpensive = false;

  cart.forEach(item => {
    // 1. Item Summary (All items)
    const summaryItem = document.createElement('li');
    summaryItem.innerHTML = `${item.name}
<span>${formatINR(item.total)}</span>`;
    itemSummaryList.appendChild(summaryItem);

    // 2. Low-Cost Items (Price < 1000)
    if (item.price < 1000) {
      hasLowCost = true;
      const lowCostItem = document.createElement('li');

```

```

        lowCostItem.innerHTML = `${item.name}
<span>${formatINR(item.price)}</span>`;
        lowCostList.appendChild(lowCostItem);
    }

    // 3. Expensive Items (Price > 1000)
    if (item.price > 1000) {
        hasExpensive = true;
        const expensiveItem = document.createElement('li');
        expensiveItem.innerHTML = `${item.name} <span>${
formatINR(item.price)}</span>`;
        expensiveList.appendChild(expensiveItem);
    }
});

if (hasLowCost) {
    noLowCostMsg.style.display = 'none';
} else {
    noLowCostMsg.style.display = 'block';
    noLowCostMsg.innerText = cart.length === 0 ? "Cart is empty." : "No items
priced below ₹1000 found.";
}

if (hasExpensive) {
    noExpensiveMsg.style.display = 'none';
} else {
    noExpensiveMsg.style.display = 'block';
    noExpensiveMsg.innerText = cart.length === 0 ? "Cart is empty." : "No items
priced above ₹1000 found.";
}
}

// Event Listeners for "Enter" key
document.addEventListener('DOMContentLoaded', () => {
    const inputs = ['productName', 'price', 'quantity'];

    inputs.forEach(id => {
        document.getElementById(id).addEventListener('keypress', function(event) {
            if (event.key === 'Enter') {
                addProduct();
            }
        });
    });

    renderCart();
    calculateTotals();
});

```

4. Output:

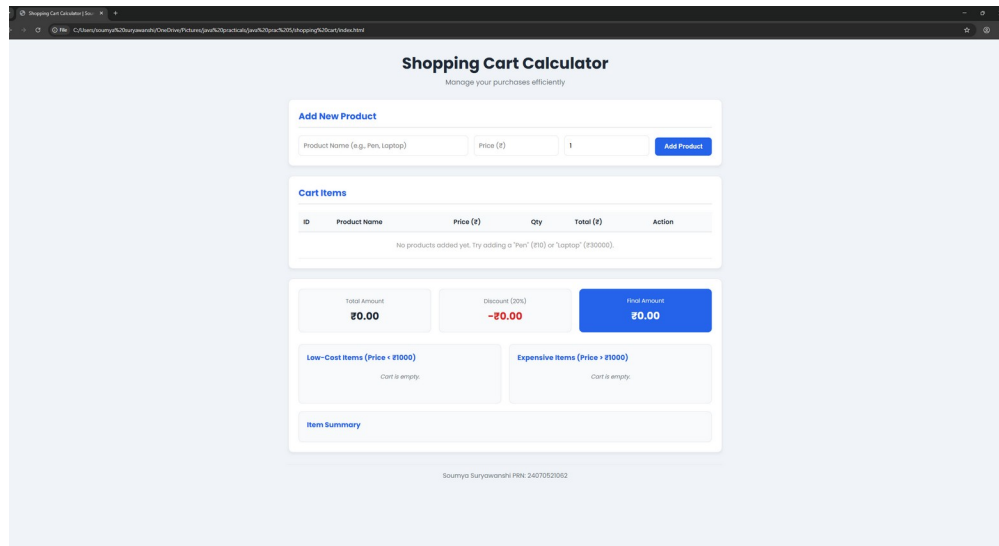


Fig 4: Shopping Cart Calculator output — PRN: 24070521062, Name: Soumya Suryawanshi, File Path: C:/Users/soumya suryawanshi/OneDrive/Pictures/java practicals/java prac 5/shopping cart/index.html

5. Case Study Title:

Class-Based Marks Tracker — Object-Oriented Array Handling for Student Performance Analysis

6. Case Study Program Code:

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Class-Based Marks Tracker</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="container">
    <header>
      <h1>Student Marks Class</h1>
      <p>Soumya suryawanshi PRN - 24070521062</p>
    </header>

    <main class="card">
      <div class="input-section">
        <div class="form-group">
          <label for="studentName">Student Name</label>
          <input type="text" id="studentName" placeholder="e.g. Alice">
        </div>
        <div class="form-group">
          <label for="studentMark">Mark (0-100)</label>
        </div>
      </div>
    </main>
  </div>
</body>
</html>
```

```

        <input type="number" id="studentMark" placeholder="e.g. 95"
min="0" max="100">
    </div>
    <button id="addBtn" class="btn-add">Add Student</button>
</div>

<div class="stats-section">
    <div class="stat-card max">
        <span class="label">Top Performer</span>
        <span id="max-name" class="value">--</span>
        <span id="max-score" class="sub-label">0</span>
    </div>
    <div class="stat-card min">
        <span class="label">Needs Improvement</span>
        <span id="min-name" class="value">--</span>
        <span id="min-score" class="sub-label">0</span>
    </div>
    <div class="stat-card average">
        <span class="label">Class Average</span>
        <span id="avg-score" class="value">--</span>
    </div>
</div>

<div class="list-section">
    <h2>Class Roster</h2>
    <div id="student-list" class="student-grid"></div>
</div>
</main>
</div>

<script src="script.js"></script>
</body>
</html>

```

script.js

```

class Student {
    constructor(name, mark) {
        this.name = name;
        this.mark = mark;
    }

    getGrade() {
        if (this.mark >= 90) return 'high';
        if (this.mark < 60) return 'low';
        return 'mid';
    }
}

class MarksManager {
    constructor() {
        this.students = []; // Array of Student objects
    }
}

```



```

    addStudent(name, mark) {
        const student = new Student(name, mark);
        this.students.push(student); // Array method: push
        return student;
    }

    getStats() {
        if (this.students.length === 0) return null;

        let maxStudent = this.students[0];
        let minStudent = this.students[0];
        let sum = 0;

        this.students.forEach(s => { // Array method: forEach
            if (s.mark > maxStudent.mark) maxStudent = s;
            if (s.mark < minStudent.mark) minStudent = s;
            sum += s.mark;
        });

        return {
            max: maxStudent,
            min: minStudent,
            average: (sum / this.students.length).toFixed(2)
        };
    }
}

const manager = new MarksManager();

const nameInput = document.getElementById('studentName');
const markInput = document.getElementById('studentMark');
const addBtn = document.getElementById('addBtn');
const studentListEl = document.getElementById('student-list');

const maxNameEl = document.getElementById('max-name');
const maxScoreEl = document.getElementById('max-score');
const minNameEl = document.getElementById('min-name');
const minScoreEl = document.getElementById('min-score');
const avgScoreEl = document.getElementById('avg-score');

addBtn.addEventListener('click', handleAdd);
nameInput.addEventListener('keypress', e => { if(e.key === 'Enter')
handleAdd(); });
markInput.addEventListener('keypress', e => { if(e.key === 'Enter')
handleAdd(); });

function handleAdd() {
    const name = nameInput.value.trim();
    const mark = parseFloat(markInput.value);

    if (!name) { alert('Enter a name'); return; }
    if (isNaN(mark) || mark < 0 || mark > 100) { alert('Enter valid mark (0-100)');
return; }

```

```

    manager.addStudent(name, mark);
    nameInput.value = '';
    markInput.value = '';
    nameInput.focus();
    updateUI();
}

function updateUI() {
    studentListEl.innerHTML = '';
    const stats = manager.getStats();

    if (!stats) {
        maxNameEl.textContent = '--'; maxScoreEl.textContent = '0';
        minNameEl.textContent = '--'; minScoreEl.textContent = '0';
        avgScoreEl.textContent = '--';
        return;
    }

    maxNameEl.textContent = stats.max.name;
    maxScoreEl.textContent = stats.max.mark;
    minNameEl.textContent = stats.min.name;
    minScoreEl.textContent = stats.min.mark;
    avgScoreEl.textContent = stats.average;

    manager.students.forEach(student => { // Array method: forEach
        const card = document.createElement('div');
        card.className = 'student-card';
        card.innerHTML = `
            <div class="student-info">
                <h3>${student.name}</h3>
                <p>Student</p>
            </div>
            <div class="mark-badge ${student.getGrade()}>${student.mark}</div>
        `;
        studentListEl.appendChild(card);
    });
}

```

7. Output:

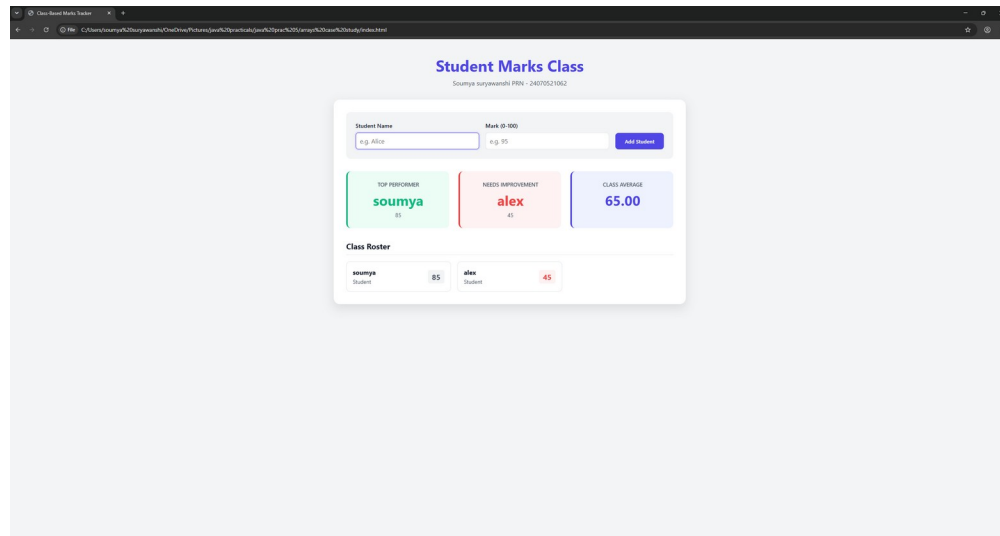


Fig 7: Class-Based Marks Tracker output — PRN: 24070521062, Name: Soumya Suryawanshi, File Path: C:/Users/soumya suryawanshi/OneDrive/Pictures/java practicals/java prac 5/arrays case study/index.html

8. Result/Conclusion:

A Shopping Cart Total Calculator was successfully designed and implemented using JavaScript array methods (push, filter, forEach, and reduce) together with object handling to store, update, and total cart items, applying a 20% discount to compute the final payable amount and dynamically classifying items into low-cost and expensive categories. In the accompanying case study, a class-based Marks Tracker was built using JavaScript classes (Student and MarksManager) to encapsulate student objects inside an array, and array iteration was used to compute the top performer, the student needing improvement, and the class average. Both implementations produced the expected output for the given inputs and reinforced practical understanding of array methods, object-oriented data modelling, and dynamic DOM rendering.

Upload the Soft copy as well as complete code on GitHub link is mandatory.